

Rapport de Projet de Fin d'Études / Projet Tuteuré

Projet MiniAws

Déploiement et Gestion de Cloud Privé par IA

Présenté par :

OUAZZOU Abdelhamid

Encadré par :

Pr. BOUSMAH



Année Universitaire : 2025 - 2026

Dédicaces

À mes parents, pour leur soutien inconditionnel et leurs sacrifices qui m'ont permis d'arriver jusqu'ici.

À ma famille et à mes amis, pour leurs encouragements continus tout au long de mon parcours universitaire.

À tous ceux qui ont cru en moi et m'ont inspiré à donner le meilleur de moi-même.

Remerciements

Je tiens tout d'abord à remercier mon encadrant, **[Nom de l'encadrant]**, pour sa disponibilité, ses conseils judicieux et son accompagnement tout au long de la réalisation de ce projet.

Mes remerciements s'adressent également à l'ensemble du corps professoral pour la qualité de l'enseignement prodigué. Enfin, je remercie toutes les personnes qui ont contribué de près ou de loin à l'aboutissement de ce travail.

Résumé

Le projet **MiniAws** vise à simplifier la gestion des infrastructures de cloud privé en proposant une interface mobile Android moderne couplée à une Intelligence Artificielle locale. Face à la complexité des tableaux de bord traditionnels, notre solution introduit le "IA Ops", permettant le provisionnement de machines virtuelles via la voix, le scan de QR Code ou un chat conversationnel.

Le système repose sur une architecture modulaire : un backend Java/Spring Boot pour la logique métier et le monitoring WebSockets, un microservice Python intégrant une architecture RAG avec ChromaDB pour optimiser les requêtes IA, et une application Android au design "Cyber-Cloud".

Mots-clés : Cloud Privé, Android, Spring Boot, Intelligence Artificielle, Architecture RAG, Provisioning.

Abstract

The **MiniAws** project aims to simplify the management of private cloud infrastructures by offering a modern Android mobile interface coupled with local Artificial Intelligence. Addressing the complexity of traditional dashboards, our solution introduces "AI Ops", enabling virtual machine provisioning via voice commands, QR code scanning, or conversational chat.

The system is built on a modular architecture : a Java/Spring Boot backend for business logic and WebSocket monitoring, a Python microservice integrating a RAG architecture with ChromaDB to optimize AI requests, and an Android application featuring a "Cyber-Cloud" design.

Keywords : Private Cloud, Android, Spring Boot, Artificial Intelligence, RAG Architecture, Provisioning.

Liste des Abréviations

API	Application Programming Interface
IA	Intelligence Artificielle
IAM	Identity and Access Management
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
STOMP	Simple Text Oriented Messaging Protocol
UI	User Interface
UX	User Experience
VM	Virtual Machine

Table des figures

1.1	Diagramme de Gantt prévisionnel du projet	4
2.1	Diagramme de Cas d'Utilisation de MiniAws	10
2.2	Diagramme de Classe Simplifié (Backend Data Domain)	11
2.3	Diagramme d'Activité : Processus de déploiement IA avec Cache Sémantique	12
3.1	Architecture technique globale de MiniAws	14
4.1	Interfaces d'Authentification et de Profil Utilisateur (Mockups)	18

Liste des tableaux

2.1	Matrice détaillée des droits et permissions par rôle	7
3.1	Environnement logiciel et outils de développement	16

Table des matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Abstract	iv
Liste des Abréviations	v
Liste des figures	vi
Liste des tableaux	vii
Introduction Générale	1
1 Contexte Général du Projet	2
1.1 Présentation du Projet	2
1.1.1 Problématique	2
1.1.2 Solution Proposée	3
1.2 Méthodologie de Travail et Planification	3
1.2.1 Méthodologie de Travail	3
1.2.2 Diagramme de Gantt	4
2 Analyse et Conception	6
2.1 Etude Fonctionnelle	6
2.1.1 Analyse des Besoins fonctionnels	6
2.1.2 Analyse des Besoins non fonctionnels	8
2.2 Etude Conceptuelle	8

2.2.1	Règles de Gestion	8
2.2.2	Diagrammes UML	9
3	Etude Technique	13
3.1	Architecture du projet	13
3.2	Technologies utilisées	14
3.2.1	Couche Frontend (Application Mobile)	14
3.2.2	Couche Backend (Serveur Principal)	15
3.2.3	Couche Intelligence Artificielle (RAG & Cache Sémantique) . .	15
3.3	Environnement de Travail	15
4	Mise en œuvre	17
4.1	Réalisation	17
4.1.1	Authentification et Profil (IAM)	17
4.1.2	Provisioning IA et Dashboard (IA Ops)	18
4.1.3	Monitoring et Gestion des VM (Compute)	18
4.2	DevOps (Implémentation)	18
4.2.1	Conteneurisation (Docker)	19
4.2.2	Orchestration (Docker Compose)	19
	Conclusion Générale et Perspectives	20
	Références / Webographie	22

Introduction Générale

Dans un monde numérique en constante évolution, la gestion des infrastructures informatiques et du Cloud Computing est devenue un enjeu majeur pour les entreprises. Si les solutions de cloud public offrent une grande flexibilité, de nombreuses organisations se tournent vers le cloud privé (comme Proxmox) pour des raisons de sécurité, de souveraineté des données et de réduction des coûts à long terme. Cependant, l'administration de ces environnements reste souvent complexe, nécessitant une expertise technique pointue et la manipulation d'interfaces lourdes et peu intuitives.

C'est dans ce contexte que s'inscrit notre projet intitulé **MiniAws**. L'objectif principal est de démocratiser et de moderniser l'accès aux ressources du cloud privé en proposant une solution complète, mobile et intelligente. En intégrant les récentes avancées en Intelligence Artificielle, et plus particulièrement les modèles de langage locaux (LLM) couplés à une architecture RAG (Retrieval-Augmented Generation), MiniAws introduit le concept d'"IA Ops". Il permet à un utilisateur de déployer et de gérer des serveurs virtuels simplement par la voix, par un chat textuel, ou par le scan d'un QR code, le tout depuis une interface Android moderne au design "Cyber-Cloud".

Le présent rapport détaille les différentes phases de réalisation de ce projet. Il est structuré en quatre chapitres principaux :

- Le **Chapitre 1** présente le contexte général du projet, la problématique soulevée ainsi que la solution proposée et la méthodologie de travail.
- Le **Chapitre 2** est consacré à l'analyse fonctionnelle et conceptuelle du système via la modélisation UML.
- Le **Chapitre 3** détaille l'étude technique, justifiant l'architecture logicielle choisie et les technologies utilisées (Spring Boot, Android, Python, ChromaDB).
- Enfin, le **Chapitre 4** illustre la phase de mise en œuvre, en présentant les interfaces réalisées et le processus de déploiement (DevOps).

Chapitre 1

Contexte Général du Projet

Introduction

La réussite de tout projet informatique repose sur une compréhension approfondie de son environnement, de ses enjeux et des contraintes qui l’entourent. Ce premier chapitre vise à poser les fondements du projet MiniAws. Nous commencerons par définir la problématique liée à la gestion des clouds privés actuels, avant d’exposer en détail notre solution. Enfin, nous présenterons la méthodologie adoptée pour mener à bien ce travail dans les délais impartis.

1.1 Présentation du Projet

1.1.1 Problématique

L’utilisation de solutions de virtualisation telles que Proxmox est courante pour gérer des parcs de machines virtuelles (VM). Cependant, ces plateformes présentent plusieurs limites pour un usage moderne et agile :

- **Complexité des interfaces :** Les tableaux de bord natifs sont conçus pour des administrateurs systèmes experts. La création, la configuration et le monitoring d’une VM requièrent de nombreuses étapes manuelles.
- **Absence de mobilité :** La gestion de l’infrastructure est généralement confinée à des postes de travail fixes via des navigateurs web lourds, limitant les interventions rapides en situation de mobilité.

- **Processus chronophages** : Le provisionnement manuel des ressources (allocation de CPU, RAM, OS) est répétitif et sujet aux erreurs humaines, manquant cruellement d'automatisation intelligente.

1.1.2 Solution Proposée

Pour répondre à ces problématiques, nous avons conçu **MiniAws**, une plateforme Fullstack innovante offrant un contrôle de l'infrastructure Cloud via un smartphone Android. Les principales valeurs ajoutées de la solution sont :

- **Approche "IA Ops"** : Intégration d'un agent conversationnel IA (basé sur Olama) permettant le déploiement de machines par commandes vocales (Speech-to-Text) ou par requêtes textuelles.
- **Performances optimisées par l'IA (RAG & Cache Sémantique)** : Mise en place d'un microservice dédié à l'IA qui mémorise les configurations de déploiement précédentes. Si un utilisateur demande une VM similaire, le système récupère instantanément la configuration via une recherche sémantique (Vector Database ChromaDB), réduisant le temps de réponse drastiquement.
- **Diversité de déploiement** : Outre l'IA, le système supporte la création de VM via des formulaires manuels classiques et via le scan de QR Codes.
- **Monitoring en temps réel et UI moderne** : Une interface Android esthétique (Cyber-Cloud / Glassmorphism) offrant des statistiques de consommation CPU/RAM en temps réel via des connexions WebSockets (STOMP).
- **Sécurité et Gestion des accès** : Implémentation d'un système IAM (Identity and Access Management) via Firebase, différenciant les droits des administrateurs et des utilisateurs standards, tout en gérant des quotas de ressources.

1.2 Méthodologie de Travail et Planification

1.2.1 Méthodologie de Travail

Pour la réalisation de ce projet, nous avons adopté la méthodologie ****Agile****, et plus précisément le framework ****Scrum****. Ce choix se justifie par la nécessité d'avoir une approche itérative et incrémentale, permettant de s'adapter rapidement aux défis

techniques rencontrés (notamment sur l'intégration des LLM et des WebSockets).

Le travail a été organisé en "Sprints" de deux semaines, chacun se terminant par une revue des fonctionnalités implémentées. Cette méthode nous a permis de livrer un prototype fonctionnel très tôt dans le cycle de développement et de l'enrichir progressivement jusqu'à la version finale.

1.2.2 Diagramme de Gantt

La planification a été modélisée à l'aide d'un diagramme de Gantt, illustrant l'ordonnancement des tâches suivantes :

1. **Phase d'Analyse et Conception** : Recueil des besoins, maquettage UI/UX, et réalisation des diagrammes UML (Cas d'utilisation, Classes).
2. **Phase Backend (Core)** : Développement de l'API REST avec Spring Boot, mise en place de l'authentification Firebase et configuration des WebSockets.
3. **Phase Intelligence Artificielle** : Développement du microservice Python, intégration du modèle LLM local, et implémentation du Cache Sémantique avec ChromaDB.
4. **Phase Frontend Mobile** : Développement de l'application Android native, intégration du design "Cyber-Cloud", et consommation des API backend.
5. **Phase d'Intégration et de Tests** : Liaison des différentes composantes, tests d'intégration, correction des bugs (ex : synchronisation des animations de monitoring) et validation fonctionnelle.

(Insérer ici la figure du Diagramme de Gantt)

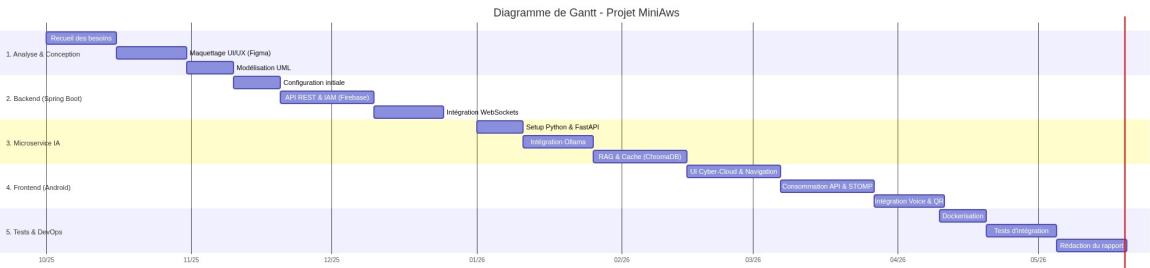


FIGURE 1.1 – Diagramme de Gantt prévisionnel du projet

Conclusion

Ce premier chapitre a permis de cerner le contexte général du projet MiniAws. Nous avons mis en évidence les limites des outils d'administration cloud classiques pour justifier notre solution mobile propulsée par l'Intelligence Artificielle. La méthodologie mise en place assure un développement rigoureux et structuré. Dans le chapitre suivant, nous aborderons l'analyse approfondie des besoins et la conception conceptuelle du système.

Chapitre 2

Analyse et Conception

Introduction

Après avoir défini le contexte et les objectifs du projet MiniAws, ce chapitre se consacre à l'analyse approfondie du système. L'étape d'analyse et de conception est cruciale car elle permet de traduire les besoins des utilisateurs en spécifications techniques claires. Nous détaillerons d'abord l'étude fonctionnelle, englobant les besoins fonctionnels et non fonctionnels. Ensuite, l'étude conceptuelle modélisera l'architecture du système à travers les diagrammes UML standard (Cas d'utilisation, Activité, Classe) et définira les règles de gestion régissant l'application.

2.1 Etude Fonctionnelle

2.1.1 Analyse des Besoins fonctionnels

Les besoins fonctionnels décrivent les actions exactes que le système MiniAws doit être capable d'exécuter. Le tableau 2.1 résume la répartition des droits d'accès selon le rôle de l'utilisateur.

Fonctionnalité	ROLE_USER	ROLE_ADMIN
Authentification (Firebase)	OUI	OUI
Déploiement VM (IA/Voix/QR/Manuel)	Propres ressources	Toutes ressources
Respect des Quotas (CPU/RAM/Nombre)	Strict	Illimité / Administrable
Monitoring Temps Réel (WebSockets)	OUI	OUI
Contrôle d'état (Start/Stop/Restart)	Propres ressources uniquement	Toutes les VMs
Suppression définitive de VM	Propres ressources uniquement	Toutes les VMs
Gestion des Profils utilisateurs	NON	OUI
Configuration globale du système	NON	OUI
Visualisation globale de l'infra	NON	OUI

TABLE 2.1 – Matrice détaillée des droits et permissions par rôle

1. Gestion de l'Identité et des Accès (IAM) :

- **Authentification** : Le système doit permettre à un utilisateur de s'inscrire, de se connecter et de se déconnecter de manière sécurisée (via Firebase Auth).
- **Gestion des Profils** : Le système doit différencier les rôles (ROLE_ADMIN et ROLE_USER).
- **Quotas** : Le système doit afficher les quotas de ressources (CPU, RAM, nombre de serveurs) alloués à chaque utilisateur sur sa page de profil.

2. Provisioning et Déploiement (IA Ops) :

- **Déploiement Vocal** : Le système doit convertir la voix de l'utilisateur en texte pour formuler une requête de création de VM.
- **Déploiement via Chatbot** : L'utilisateur doit pouvoir interagir avec un Agent IA local (Ollama) pour configurer et déployer une VM.
- **Déploiement par QR Code** : Le système doit permettre le scan d'un QR Code contenant les spécifications d'une VM, suivi d'une carte de confirmation avant le déploiement.
- **Déploiement Manuel** : L'utilisateur doit pouvoir saisir manuellement les caractéristiques techniques de la VM à créer.

3. Gestion et Monitoring des Ressources (Compute) :

- **Liste des Serveurs** : L'application doit afficher la liste des VM, avec un filtrage par statut (TOUT, RUNNING, STOPPED).
- **Contrôle de l'État** : L'utilisateur doit pouvoir Démarrer (Start), Arrêter

- (Stop) ou Supprimer (Delete) une VM spécifique.
- **Monitoring Temps Réel** : L'application doit afficher les métriques de consommation (CPU, RAM) en temps réel avec des animations fluides.
- **Détails des VM** : Le système doit fournir une vue détaillée des spécifications de chaque VM (ID, OS, configuration).

2.1.2 Analyse des Besoins non fonctionnels

Les besoins non fonctionnels définissent les critères de qualité, de performance et de sécurité du système :

- **Performance (IA)** : Grâce à l'architecture RAG et au Cache Sémantique, les requêtes IA récurrentes doivent obtenir une réponse instantanée, évitant le temps de génération complet du modèle LLM.
- **Réactivité (UI)** : L'interface Android doit être fluide, avec des transitions rapides (animées à 300ms environ) et des jauges de monitoring sans effet de clignotement.
- **Design et Ergonomie** : L'application doit adopter un thème "Cyber-Cloud" avec des effets visuels (Glassmorphism, couleurs néon) pour offrir une expérience utilisateur premium.
- **Sécurité** : La communication entre le client mobile, le backend Spring Boot et le microservice IA doit être sécurisée. Les données des utilisateurs doivent être protégées.
- **Disponibilité (Monitoring)** : La connexion WebSocket pour le monitoring doit être robuste et se reconnecter automatiquement en cas de perte de signal.

2.2 Etude Conceptuelle

2.2.1 Règles de Gestion

Pour garantir la cohérence des données et le respect de la logique métier, plusieurs règles de gestion ont été établies :

1. **RG1 - Isolation des Ressources** : Un utilisateur de rôle `ROLE_USER` ne peut voir, modifier ou supprimer que les machines virtuelles dont il est le propriétaire.

Un `ROLE_ADMIN` a une vue globale.

2. **RG2 - Limite de Quotas** : Un utilisateur ne peut pas déployer une nouvelle VM si la somme des ressources demandées (CPU, RAM) ou le nombre total de serveurs dépasse les quotas définis dans son profil.
3. **RG3 - Cache Sémantique** : Lors d'une requête de déploiement IA, le système vérifie d'abord le cache via le microservice Python (ChromaDB). Si un score de similarité supérieur à 0.9 est trouvé, la configuration en cache est utilisée. Sinon, l'agent Ollama est invoqué.
4. **RG4 - Validation des Actions Critiques** : Toute demande de suppression d'une VM doit faire l'objet d'une confirmation explicite par l'utilisateur.

2.2.2 Diagrammes UML

Afin de visualiser l'architecture et les interactions du système, nous avons utilisé le langage de modélisation UML (Unified Modeling Language).

Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation modélise les interactions entre les acteurs (Utilisateur standard, Administrateur, Système IA) et le système MiniAws. Les principaux cas d'utilisation incluent l'authentification, la consultation du tableau de bord, le déploiement de VM (par IA, voix, QR, manuel), et la gestion du cycle de vie des VM.

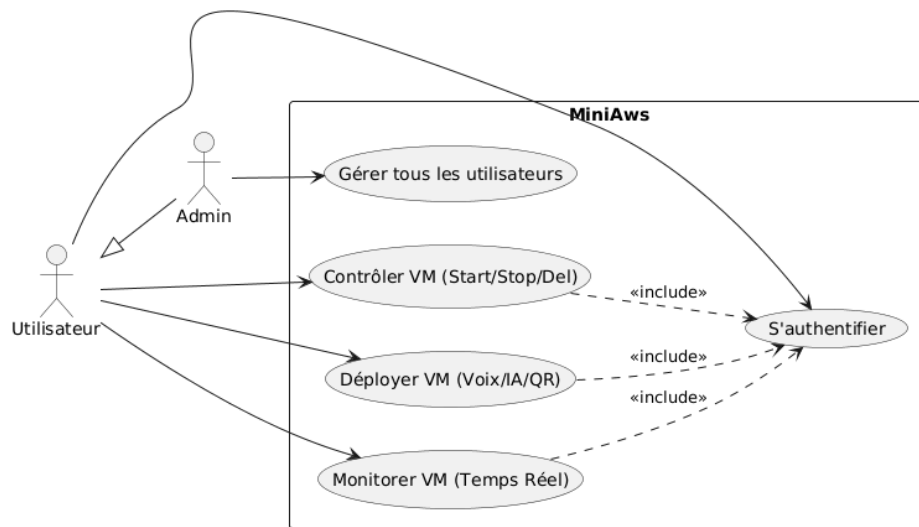


FIGURE 2.1 – Diagramme de Cas d’Utilisation de MiniAws

Diagramme de Classe

Le diagramme de classe présente la structure statique du domaine de l’application côté Backend. Il illustre les entités principales telles que `User`, `VirtualMachine`, `Metrics`, et les relations entre elles. Par exemple, un `User` possède plusieurs `VirtualMachine`, et chaque `VirtualMachine` est associée à des objets de `Metrics` pour le monitoring.

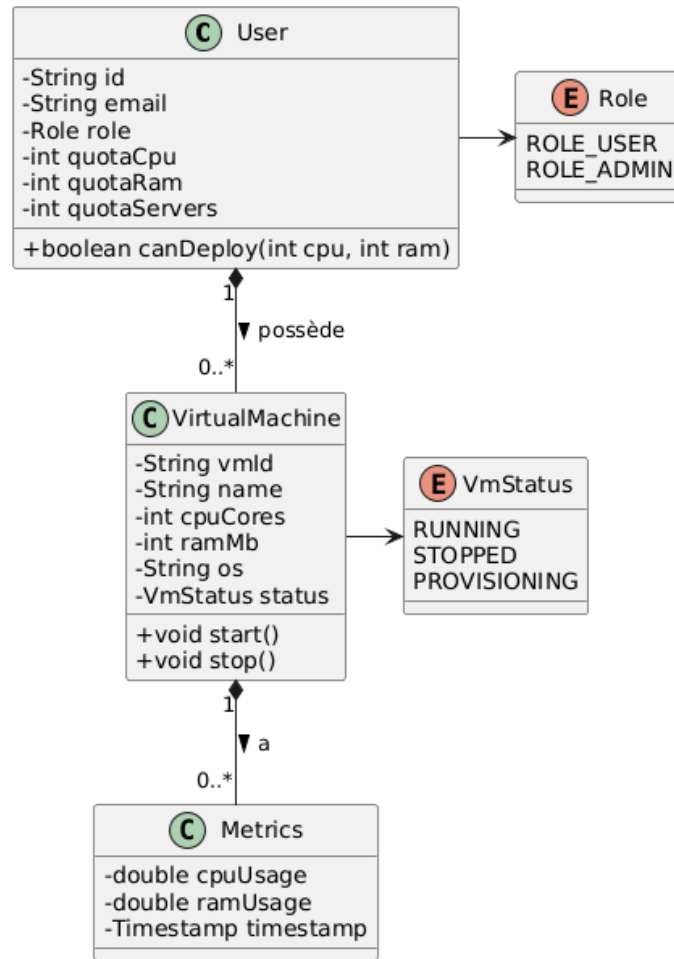


FIGURE 2.2 – Diagramme de Classe Simplifié (Backend Data Domain)

Diagramme d'Activité

Nous avons modélisé le processus de déploiement d'une VM par IA, qui illustre le mécanisme de Cache Sémantique (RAG). Le diagramme trace le chemin de la requête depuis l'application mobile jusqu'au Backend, la vérification dans ChromaDB par le microservice Python, la possible invocation du modèle Ollama, puis le retour de la configuration.

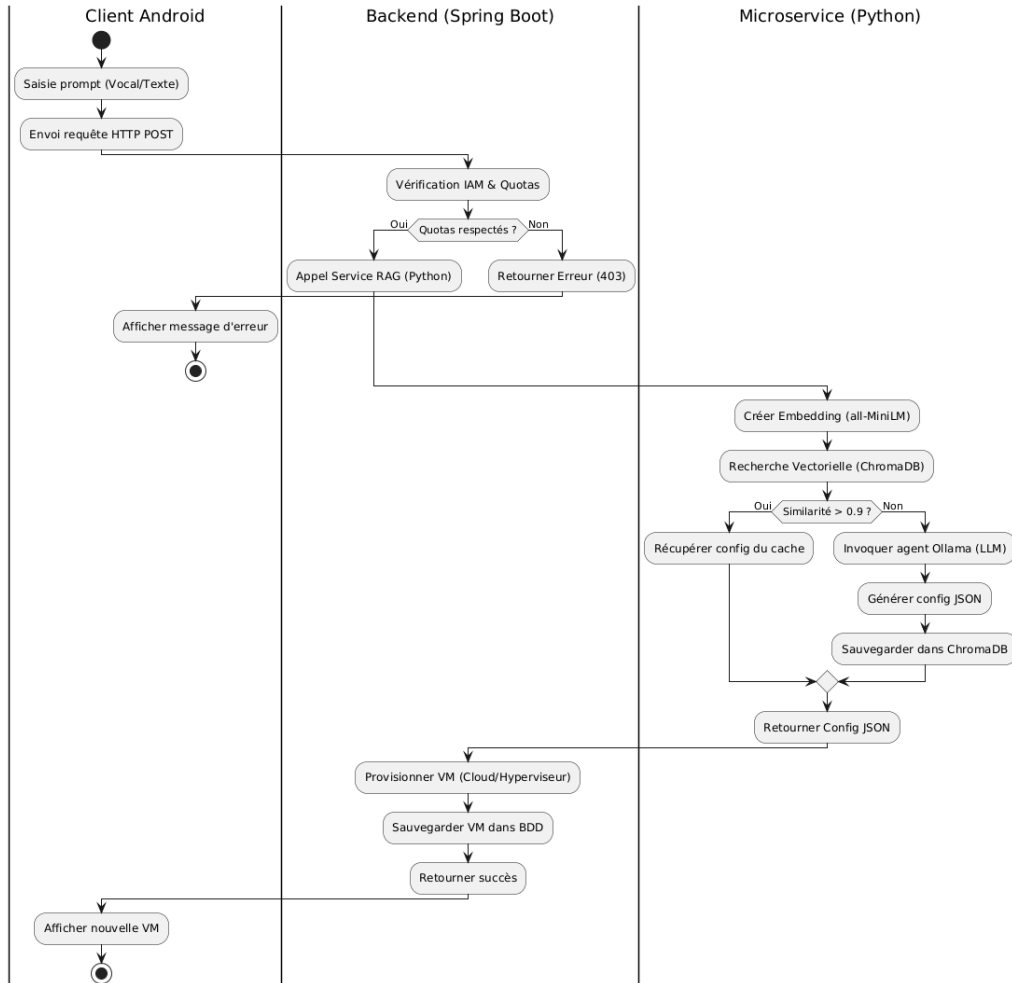


FIGURE 2.3 – Diagramme d’Activité : Processus de déploiement IA avec Cache Sémantique

Conclusion

Dans ce chapitre, nous avons détaillé les spécifications fonctionnelles et non fonctionnelles de la plateforme MiniAws. L’étude conceptuelle, soutenue par les diagrammes UML et les règles de gestion, nous a permis de structurer la logique métier et les interactions du système de manière claire. Cette modélisation solide nous permet maintenant d’aborder la phase de conception technique et le choix des technologies, qui feront l’objet du chapitre suivant.

Chapitre 3

Etude Technique

Introduction

Après avoir modélisé le système d'un point de vue conceptuel et fonctionnel, ce troisième chapitre est dédié à l'étude technique. Il s'agit d'une étape déterminante où nous définissons l'architecture globale de l'application et justifions les choix technologiques effectués pour répondre aux exigences fixées. Nous présenterons les différentes couches de notre solution, les frameworks utilisés pour le frontend, le backend et le module d'intelligence artificielle, ainsi que notre environnement de travail.

3.1 Architecture du projet

Pour garantir la scalabilité, la maintenabilité et les performances du projet **MiniAws**, nous avons opté pour une architecture distribuée, s'articulant autour de trois composants principaux :

1. **Le Client Mobile (Frontend)** : Une application Android native qui gère l'interface utilisateur, la capture audio (Speech-to-Text), le scan de QR Code, et l'affichage des métriques en temps réel via WebSocket.
2. **Le Serveur Principal (Backend)** : Développé en Spring Boot, il expose une API RESTful sécurisée. Il gère la logique métier, l'authentification (via Firebase), la gestion des ressources (IAM, Compute), et fait office de relais (Gateway) entre l'application mobile et l'hyperviseur Cloud ou le service IA.
3. **Le Microservice IA (Service RAG)** : Un service indépendant développé

en Python. Il héberge la logique d'Intelligence Artificielle, communique avec le LLM local (Ollama), et gère le Vector Store (ChromaDB) pour la recherche de similarité sémantique (Semantic Cache).

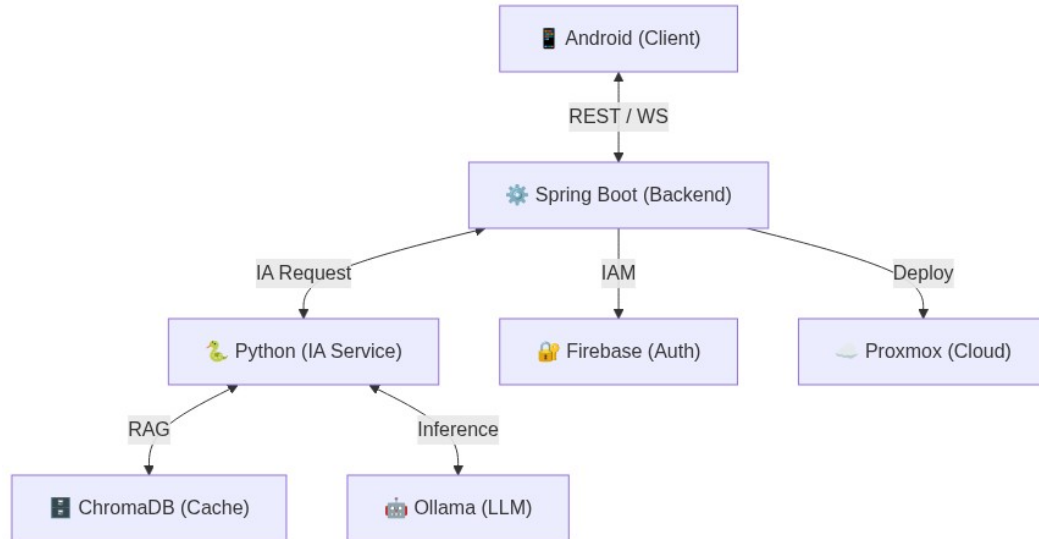


FIGURE 3.1 – Architecture technique globale de MiniAws

3.2 Technologies utilisées

Le choix des technologies s'est porté sur des outils modernes et robustes, adaptés à chaque couche de notre architecture.

3.2.1 Couche Frontend (Application Mobile)

- **Android (Java 11)** : Le SDK Android natif a été choisi pour exploiter au mieux les capacités matérielles du smartphone (Microphone pour la voix, Caméra pour les QR Codes) et garantir des animations fluides (UI Cyber-Cloud).
- **OkHttp / Retrofit** : Utilisés pour la consommation de l'API REST du backend.
- **STOMP Over WebSocket** : Protocole utilisé pour recevoir les flux de données (métriques CPU/RAM) en temps réel.

3.2.2 Couche Backend (Serveur Principal)

- **Java & Spring Boot** : Framework de référence pour le développement d'applications d'entreprise robustes. Le projet suit une architecture monolithique modulaire (modules : `iam`, `compute`, `ai_ops`, `monitoring`), facilitant la séparation des responsabilités.
- **Spring Security & Firebase Admin SDK** : Utilisés conjointement pour sécuriser les endpoints API à l'aide de tokens JWT générés par Firebase lors de l'authentification client.
- **Spring WebSocket** : Pour la mise en place du canal de communication bidirectionnel nécessaire au monitoring.

3.2.3 Couche Intelligence Artificielle (RAG & Cache Sémantique)

- **Python & FastAPI** : Langage de prédilection pour l'IA, couplé à FastAPI pour exposer des points de terminaison performants et asynchrones.
- **LangChain** : Framework facilitant l'orchestration des requêtes vers les modèles de langage et la gestion des prompts.
- **ChromaDB** : Base de données vectorielle intégrée localement pour stocker les embeddings. L'embedding est généré localement via le modèle `all-MiniLM-L6-v2` pour optimiser le cache sémantique.
- **Ollama** : Moteur d'exécution local pour les modèles de langage (LLM), garantissant la confidentialité des données et l'absence de coûts d'API cloud.

3.3 Environnement de Travail

Pour mener à bien ce projet, nous avons utilisé un ensemble d'outils favorisant la productivité et la collaboration. Le tableau 3.1 détaille les outils et versions utilisés.

Composant	Outil / Version
Système d'Exploitation	Windows 11 / Linux (Ubuntu)
IDE Frontend	Android Studio Jellyfish
IDE Backend	IntelliJ IDEA 2024.1
IDE IA	VS Code / PyCharm
Langages	Java 11, Python 3.10, XML
Gestion de versions	Git / GitHub
Conteneurisation	Docker & Docker Compose
Tests API	Postman

TABLE 3.1 – Environnement logiciel et outils de développement

Nous détaillons ci-dessous les spécificités de ces environnements :

- **Environnements de Développement Intégré (IDE) :**
 - *Android Studio* pour le développement mobile.
 - *IntelliJ IDEA* pour le développement du backend Spring Boot.
 - *PyCharm* / *VS Code* pour le microservice Python.
- **Gestion de versions : Git** et **GitHub/GitLab** pour le suivi des modifications et la sauvegarde du code source.
- **Tests d'API : Postman** pour simuler et valider les requêtes HTTP (REST) ainsi que les connexions WebSocket en phase de développement.
- **Outils de modélisation : StarUML / Draw.io** pour la création des diagrammes UML et de l'architecture.

Conclusion

L'étude technique détaillée dans ce chapitre a permis de figer l'écosystème technologique du projet. L'architecture modulaire adoptée, séparant l'interface utilisateur, la logique métier centralisée et le traitement intelligent, offre des garanties de performance et d'évolutivité. Les choix technologiques étant validés, le chapitre suivant sera consacré à la mise en œuvre concrète du projet, en illustrant la réalisation des interfaces et les aspects de déploiement (DevOps).

Chapitre 4

Mise en œuvre

Introduction

Ce dernier chapitre concrétise les concepts et les choix techniques définis précédemment. Nous y présenterons la phase de réalisation de l'application MiniAws à travers ses interfaces principales, illustrant l'expérience utilisateur (UX) et le design "Cyber-Cloud". Dans un second temps, nous aborderons la dimension DevOps du projet, détaillant les méthodes de déploiement et d'orchestration des différents services qui composent notre infrastructure.

4.1 Réalisation

La réalisation de l'interface client a été pensée pour offrir une expérience fluide, immersive et intuitive. Voici un aperçu des fonctionnalités clés implémentées :

4.1.1 Authentification et Profil (IAM)

Le module de sécurité permet un accès contrôlé à l'application.

- **Page de Connexion/Inscription** : Interfaces épurées connectées à Firebase Authentication.
- **Page Profil** : Affiche les informations de l'utilisateur, son rôle (User/Admin) et des jauges visuelles représentant ses quotas actuels (CPU, RAM, nombre de serveurs).

(Insérer ici les captures d'écran des interfaces Auth et Profil)



FIGURE 4.1 – Interfaces d'Authentification et de Profil Utilisateur (Mockups)

4.1.2 Provisioning IA et Dashboard (IA Ops)

Le tableau de bord est le cœur du contrôle de l'infrastructure.

- **Chat IA et Commande Vocale :** Une interface de messagerie (bulles "néon/glass") permet à l'utilisateur de discuter avec l'agent Ollama. Un bouton de commande vocale active le Speech-to-Text.
- **Scan de QR Code :** Permet de scanner des configurations prédéfinies, suivi de l'affichage d'une carte de confirmation (Glassmorphism) avant d'initier le déploiement.

(Insérer ici les captures d'écran du Chat IA et du scan QR Code)

4.1.3 Monitoring et Gestion des VM (Compute)

- **Liste des Serveurs :** Affichage sous forme de cartes dynamiques avec des options de filtrage rapide (Running/Stopped).
- **Monitoring en Temps Réel :** Barres de progression indiquant l'usage CPU/RAM, mises à jour en direct via STOMP. L'interface "Cyber-Cloud" s'appuie sur des composants 100% natifs (ex : `bg_glass_card.xml`, `bg_neon_button.xml`) garantissant une fluidité maximale (300ms) sans surcharger la mémoire avec des images.
- **Détails et Contrôle :** Une vue détaillée permet d'envoyer des commandes de démarrage, d'arrêt ou de suppression de la machine virtuelle.

(Insérer ici les captures d'écran de la liste des serveurs et du monitoring)

4.2 DevOps (Implémentation)

L'approche DevOps a été intégrée pour garantir un déploiement fiable et reproductible des composants côté serveur.

4.2.1 Conteneurisation (Docker)

Afin d'isoler les environnements et de faciliter le déploiement croisé, nous avons conteneurisé nos applications.

- **Backend Spring Boot** : Emballé dans une image Docker basée sur OpenJDK 17.
- **Microservice Python (IA)** : Conteneurisé avec les dépendances FastAPI, LangChain et ChromaDB isolées dans un environnement virtuel.

4.2.2 Orchestration (Docker Compose)

Nous utilisons `docker-compose` pour orchestrer l'ensemble de la stack backend. Le fichier `docker-compose.yml` (présent dans le dossier racine `miniaws`) permet de lancer simultanément :

- Le serveur Spring Boot (port 8080).
- Le microservice IA Python.
- Les services annexes si nécessaires (base de données relationnelle, Ollama local s'il est exécuté en conteneur).

Cette configuration garantit que les services communiquent entre eux via un réseau Docker interne sécurisé.

Conclusion

Ce chapitre a mis en lumière la phase de réalisation de MiniAws. Les interfaces développées démontrent l'aboutissement de nos choix ergonomiques (Cyber-Cloud), offrant une gestion des machines virtuelles à la fois puissante et accessible. De plus, la démarche DevOps mise en place assure une gestion efficace du cycle de vie des services backend, de leur construction à leur exécution via Docker.

Conclusion Générale et Perspectives

Le projet **MiniAws** a été l'occasion de concevoir et de réaliser une solution novatrice pour la gestion d'infrastructures de cloud privé. Face à la lourdeur des outils d'administration traditionnels, notre objectif était d'apporter mobilité, simplicité et intelligence.

La réussite de ce projet s'articule autour de plusieurs réalisations majeures :

- La mise en place d'une **architecture modulaire et sécurisée**, séparant clairement les responsabilités entre le Frontend mobile, le Backend Java/Spring Boot, et le Microservice IA en Python.
- L'intégration du concept d'**IA Ops**, permettant de piloter des déploiements complexes via des commandes vocales ou un agent conversationnel (Ollama), le tout optimisé par un Cache Sémantique (ChromaDB) qui réduit drastiquement les temps de réponse.
- La conception d'une application **Android native de haute qualité**, dotée d'un design "Cyber-Cloud" immersif et proposant un monitoring en temps réel via des connexions WebSockets fluides.

Ce projet de fin d'études nous a permis de consolider nos acquis académiques et d'explorer des technologies de pointe (LLM local, RAG, WebSockets, conteneurisation Docker).

Perspectives d'évolution : Bien que l'application réponde aux exigences initiales, plusieurs axes d'amélioration peuvent être envisagés pour les futures versions :

- **Graphes Historiques :** Intégrer une vue graphique affichant l'historique des métriques (CPU/RAM/Réseau) sur plusieurs heures ou jours.
- **Orchestration Kubernetes :** Évoluer d'un déploiement `docker-compose` vers un cluster Kubernetes (K8s) pour assurer la haute disponibilité et l'auto-scaling des microservices.

- **Support Multi-Hyperviseurs :** Étendre la compatibilité du backend (actuellement orienté Proxmox/simulations) pour supporter d'autres environnements tels que VMware ESXi ou OpenStack.

Références / Webographie

- Spring Boot Documentation : <https://spring.io/projects/spring-boot>
- Android Developers : <https://developer.android.com/>
- LangChain Python : <https://python.langchain.com/>
- ChromaDB : <https://www.trychroma.com/>
- Firebase Authentication : <https://firebase.google.com/docs/auth>
- Ollama - Local LLMs : <https://ollama.com/>